

BEAM.SMP: From Single-Core Isolation to Mandatory Symmetric Multiprocessing

Namdak Tonpa Norbu Ketaka

August 11, 2026

Abstract

Erlang/OTP was designed from the outset for massive concurrency through lightweight processes and message passing. For more than a decade the runtime system existed in two parallel forms: a single-core (non-SMP) emulator and a SMP emulator. In OTP 21 the non-SMP runtime was permanently removed. This article examines the two architectures, the engineering and economic motivations for the transition, and the concrete consequences for core subsystems including the ASN.1 compilation, drivers, and native-code interfaces. The analysis is framed for systems that require a modern SMP runtime, particularly in high-assurance and bare-metal environments such as seL4.

Contents

1	Introduction	2
2	Architectural Comparison	2
2.1	Single-core (non-SMP) architecture (OTP ≤ 20)	2
2.2	Symmetric Multiprocessing architecture (OTP 21+)	2
3	Motivation for the Transition and Removal	3
3.1	Maintenance cost	3
3.2	Dirty schedulers	3
3.3	Hardware reality	3
4	Consequences for Core Subsystems	3
4.1	Cryptographic application (<code>crypto</code>)	3
4.2	ASN.1 compiler and runtime	4
4.3	Drivers and the I/O subsystem	4
4.4	Native Implemented Functions (NIFs)	4
4.5	Memory management and atomics	4
4.6	Implications for High-Assurance and Bare-Metal Deployments . .	4
5	Conclusion	5

1 Introduction

Erlang’s process model is fundamentally sequential from the point of view of a single process: a process executes a sequence of reductions until it is suspended. Early implementations mapped this model onto a single operating-system thread. With the arrival of multi-core commodity hardware the OTP team introduced an SMP runtime (first stable in OTP R11B, 2006) that retained the same language-level semantics while allowing multiple schedulers to run Erlang processes in parallel.

For twelve years both runtimes were maintained. In April 2017 the OTP team announced the deprecation of the non-SMP emulators; they were removed entirely in OTP 21 (2018). From that release onward every supported Erlang system is an SMP system, even when started with a single scheduler (`+S 1`).

2 Architectural Comparison

2.1 Single-core (non-SMP) architecture (OTP $\leq$ 20)

- One scheduler thread executes all Erlang processes.
- No operating-system threads are required for the core emulator (`--disable-threads` was legal).
- Global data structures (atom table, process table, ETS meta-data, binary heap) need only single-threaded mutual exclusion or none at all.
- Garbage collection is strictly sequential.
- Drivers and NIFs run to completion on the single scheduler; long-running native code blocks the entire virtual machine.
- The resulting binary can be linked against a minimal C library (newlib) and run as a single thread of control on a bare-metal or capability-based kernel such as seL4.

2.2 Symmetric Multiprocessing architecture (OTP 21+)

1. N normal schedulers (default = number of logical cores).
2. Separate dirty CPU and dirty I/O scheduler pools for long-running or blocking native code.
3. Auxiliary OS threads for timers, signals, asynchronous drivers and system management.
4. Lock-free and fine-grained locking protocols for run queues, process migration, message passing and memory allocators.
5. Concurrent garbage collection of process heaps and the shared binary heap.

6. Mandatory thread support at build time; the configure options that produced a pure single-threaded emulator were deleted.

The language-level semantics remain identical: processes do not share mutable data and communicate only by message passing. The difference lies entirely in the runtime realisation of that model.

3 Motivation for the Transition and Removal

Three interlocking reasons drove the decision.

3.1 Maintenance cost

Every new feature—dirty schedulers, true asynchronous signalling, improved ETS concurrency, the later JIT—had to be implemented and tested twice. The non-SMP path became a permanent tax on development velocity.

3.2 Dirty schedulers

Long-running NIFs had always been a latent reliability problem. Dirty schedulers solve it, but only in an SMP setting. Implementing an equivalent mechanism for the non-SMP emulator was judged disproportionately expensive.

3.3 Hardware reality

By 2017 single-core performance had stagnated while core counts continued to rise. The performance advantage once claimed for the non-SMP emulator on certain workloads had largely disappeared, and the operational simplicity of a single binary that automatically exploits available cores became more valuable than the residual single-threaded optimisations.

Consequently the non-SMP emulators were deprecated in OTP 20 and removed in OTP 21. The only remaining way to obtain single-scheduler behaviour is the run-time flag `+S 1`, which still executes inside the SMP codebase and retains the auxiliary threads required by that codebase.

4 Consequences for Core Subsystems

4.1 Cryptographic application (crypto)

OTP 20 linked against OpenSSL (or LibreSSL) through a set of NIFs that assumed a single-threaded caller. In the SMP world the same NIFs must be re-entrant or protected by locks; OpenSSL itself must be initialised for multi-threaded use (`CRYPTO_set_locking_callback` in 1.0.x, or the newer thread-safe defaults in 1.1+/LibreSSL). Static linking of a carefully configured LibreSSL remains feasible on both architectures, but the bare-metal single-core build can

use a simpler, non-threaded LibreSSL configuration, while the SMP build must guarantee thread safety and supply entropy from multiple schedulers.

4.2 ASN.1 compiler and runtime

The ASN.1 application is largely unaffected at the language level. The generated Erlang code is pure and therefore safe on both runtimes. The difference appears in the supporting C code used by the PER/BER codecs when linked as NIFs or linked-in drivers: any global state must be made thread-local or protected under SMP. In practice the OTP ASN.1 implementation already satisfied these requirements by the time of the transition.

4.3 Drivers and the I/O subsystem

Classic drivers ran exclusively on the single scheduler. Under SMP they execute on a scheduler thread and may be migrated; the driver API therefore gained explicit thread-safety obligations. The older asynchronous-thread pool (+A) was largely superseded by dirty I/O schedulers. A frozen OTP 20 bare-metal build can retain the simple synchronous driver model; an OTP 29 SMP build on seL4 must map dirty schedulers and the remaining async threads onto seL4 threads or notification objects.

4.4 Native Implemented Functions (NIFs)

The NIF API acquired the ability to schedule work on dirty schedulers. A NIF written for OTP 20 may still run on a modern system, but a long-running NIF will block a scheduler and degrade latency. The recommended style therefore changed from “finish quickly or accept the consequences” to “off-load to a dirty scheduler.”

4.5 Memory management and atomics

The single-core runtime could use ordinary C loads and stores for many internal structures. The SMP runtime relies on C11 atomics (or compiler built-ins) and carefully designed lock-free algorithms. This is the principal reason a pure newlib/bare-metal port becomes substantially harder after OTP 20: the required atomic primitives and memory model must be supplied by the underlying platform or by a thin compatibility layer.

4.6 Implications for High-Assurance and Bare-Metal Deployments

Organisations that maintain a frozen OTP 20 single-core tree for seL4 or other minimal environments gain a dramatically smaller trusted computing base: one thread, no OS-thread synchronisation inside the emulator, and a cryptographic library that need not be thread-aware. The modern SMP tree, while mandatory

for any system that must scale beyond a single core, imposes a richer set of requirements on the host (threads, notifications, atomics, multi-core memory consistency). The two architectures are therefore best treated as distinct products with a common language front-end rather than as two configurations of a single runtime.

5 Conclusion

The removal of the non-SMP emulator in OTP 21 marked the end of an era in which Erlang could be regarded as an essentially sequential abstract machine that happened to support concurrency. From that release onward the runtime is a genuine multi-core virtual machine whose sequential appearance is an emergent property of the language, not of the implementation. The transition was driven by maintenance economics, the necessity of dirty schedulers, and the realities of contemporary hardware. Subsystems such as `crypto`, ASN.1, drivers and NIFs adapted with varying degrees of visible change; the deepest impact is felt in the memory model and the threading assumptions that any new port must satisfy. For high-assurance single-core systems the frozen OTP 20 architecture remains a rational and defensible choice; for everything else the SMP runtime is the only supported path.

References

- [1] L. Larsson. Deprecation and removal of the non-smp emulator. Erlang Questions mailing list, April 2017.
- [2] Erlang/OTP 21.0 Release Notes, OTP-14518.
- [3] K. Lundin et al. Historical accounts of SMP development. Erlang User Conference presentations, 2008–2010.
- [4] Official OTP documentation on schedulers, dirty schedulers and the NIF API (OTP 24–29).